

JavaScript Scopes

[Back to JS page](#)

Table of Contents

- [JavaScript Scopes](#)
 - [Global Scope](#)
 - [Example 1](#)
 - [Function Scope](#)
 - [Example 2](#)
 - [Block Scope](#)
 - [Example 3](#)
 - [Automatically Global](#)
 - [Example 4](#)
 - [The Lifetime of JavaScript Variables](#)
 - [Example 5](#)
 - [Document](#)
 - [Reference](#)

Global Scope

JavaScript has **function scope**: each function creates a new scope.

Global scope means that variables declared outside any function are accessible from anywhere in the program:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Global Scope</h4>
<p id="demo"></p>

<script>
// Global variable - accessible everywhere
let carName = "Volvo";

function myFunction() {
  // Can access global variable inside a function
  return "Inside function: " + carName;
}

document.getElementById("demo").innerHTML =
  "Outside function: " + carName + "<br>" +
  myFunction();
</script>

</body>
</html>
```

Example 1

Result [View Example](#)

Function Scope

Function scope means that variables declared inside a function are only accessible within that function. They cannot be accessed outside:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Function Scope</h4>
<p id="demo"></p>

<script>
function myFunction() {
  // Local variable - only accessible inside this function
  let carName = "Volvo";
  return "Inside function: " + carName;
}

document.getElementById("demo").innerHTML =
  myFunction() + "<br><br>" +
  "Trying to access carName outside would cause an error";
</script>

</body>
</html>
```

Example 2

Result [View Example](#)

Block Scope

Block scope means that variables declared with `let` and `const` inside a block `{ }` are only accessible within that block:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Block Scope</h4>
<p id="demo"></p>

<script>
{
  // Block scope - only accessible inside this block
  let x = 2;
  const y = 3;
  var z = 5;  // var is NOT block scoped
}

// x and y are NOT accessible here
// z IS accessible here because var is function scoped

document.getElementById("demo").innerHTML =
  "z (var - accessible): " + z + "<br>" +
  "x (let - NOT accessible outside block)" + "<br>" +
  "y (const - NOT accessible outside block)";
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Automatically Global

If you assign a value to a variable that has not been declared, it will automatically become a **global** variable:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Automatically Global</h4>
<p id="demo"></p>

<script>
function myFunction() {
  // This becomes GLOBAL automatically (not recommended!)
  carName = "Volvo";
}

// Call the function first
myFunction();

// Now carName is global and accessible here
document.getElementById("demo").innerHTML =
  "After running myFunction():<br>" +
  "carName = " + carName + "<br><br>" +
  "Warning: This variable is now global!<br>" +
  "Always declare variables with let/const/var";
</script>

</body>
</html>
```

Example 4

Result [View Example](#)

The Lifetime of JavaScript Variables

The lifetime of a JavaScript variable starts when it is declared. Global variables live until the page is closed. Local variables are deleted when the function completes:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Scopes</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>The Lifetime of JavaScript Variables</h4>
<p id="demo"></p>

<script>
// Global variable – lives until page is closed
let globalVar = "I live as long as the page";

function counterExample() {
  // Local variable – created each time function runs
  // Deleted when function completes
  let localVar = "I only live inside the function";
  return localVar;
}

// Call the function to see local variable
let result = counterExample();

let text = "Global variable: " + globalVar + "<br>";
text += "Local variable (from function): " + result + "<br><br>";
text += "Global variables: live until page is closed<br>";
text += "Local variables: deleted after function completes<br>";
text += "Each function call creates new local variables";

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

Example 5

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript Scopes](#)